Task: T25
Name: Nigel James Noby
Roll Number: 25

1. Aim

To monitor TCP packets using a raw packet socket, manually read the Ethernet, IPv4, and TCP header fields, and log destination ports that are outside the same allowed-port list used in T24.

2. Course Outcome and Module

CO3: Monitor outgoing TCP connections, flag suspicious attempts such as non-standard ports, and log them to a file.

3. Tools and Versions

Tool	Version / requirement
Operating system	Linux with AF_PACKET support; Windows uses a different SIO_RCVALL approach and needs a separate platform-specific program.
Python	Python 3.8 or later
Libraries	Python standard library: socket, struct, datetime
Privileges	Root/sudo on Linux
Test utility	Netcat (nc) or an equivalent TCP client; use localhost only

4. Program or Commands

sniffer_raw.py

```
import socket
import struct
from datetime import datetime

ALLOWED_PORTS = {22, 53, 80, 443}
LOG_FILE = "suspicious_raw.log"
ETH_P_ALL = 0x0003

def handle_frame(frame):
    # Ethernet header: 14 bytes. EtherType is bytes 12-13.
    if len(frame) < 14:
        return

    ethertype = struct.unpack("!H", frame[12:14])[0]

    # 0x0800 means IPv4.
    if ethertype != 0x0800 or len(frame) < 34:
        return

    ip_start = 14
    version_ihl = frame[ip_start]
    version = version_ihl >> 4
    ip_header_len = (version_ihl & 0x0F) * 4
    if version != 4 or ip_header_len < 20:
        return
    if len(frame) < ip_start + ip_header_len:
        return
```

```
    protocol = frame[ip_start + 9]
    # Protocol number 6 means TCP.
    if protocol != 6:
        return

    src_ip = socket.inet_ntoa(frame[ip_start + 12:ip_start + 16])
    dst_ip = socket.inet_ntoa(frame[ip_start + 16:ip_start + 20])

    tcp_start = ip_start + ip_header_len
    if len(frame) < tcp_start + 20:
        return

    src_port, dst_port = struct.unpack("!HH", frame[tcp_start:tcp_start + 4])
    if dst_port not in ALLOWED_PORTS:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        message = (
            f"{timestamp} | source={src_ip}:{src_port} | "
            f"destination={dst_ip}:{dst_port}\n"
        )
        print(message, end="")
        with open(LOG_FILE, "a", encoding="utf-8") as log:
            log.write(message)

def main():
    # AF_PACKET is available on Linux. It receives Ethernet frames.
    sock = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
                         socket.htons(ETH_P_ALL))
    print("Monitoring raw Ethernet frames. Press Ctrl+C to stop.")
    print("Allowed destination ports:", sorted(ALLOWED_PORTS))
    try:
        while True:
            frame, address = sock.recvfrom(65535)
            handle_frame(frame)
    except KeyboardInterrupt:
        print("\nStopped.")
    finally:
        sock.close()

if __name__ == "__main__":
    main()
```

5. How to Run

Linux (Ubuntu/Kali):

```
sudo python3 sniffer_raw.py
```

Keep it running, then in another terminal run the localhost test:

```
nc -vz 127.0.0.1 4444
```

Stop the sniffer with Ctrl+C. Review suspicious_raw.log in the directory where the program was run.

Important platform note: AF_PACKET is Linux-specific and is not available on macOS or standard Windows Python sockets. On macOS, the lab must use a supported alternative capture mechanism and verify it explicitly. On Windows, a raw-capture version generally uses a Windows-specific mechanism such as SIO_RCVALL with appropriate privileges; do not claim this Linux program runs unchanged on those systems.

6. Sample Output with Screenshots



Figure T25-1: Output from the raw-socket sniffer.



Figure T25-2: Contents of suspicious_raw.log after a successful capture.

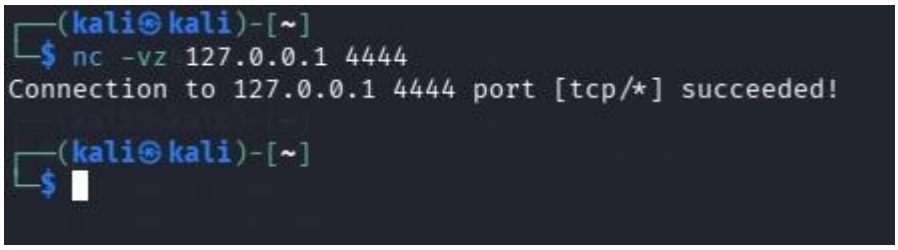


Figure T25-3: Localhost TCP test to port 4444 (nc -vz).

7. Result

The raw-socket program was prepared to receive Ethernet frames, identify IPv4 packets carrying TCP, extract source/destination IP addresses and TCP ports, and log destination ports outside the allowed list. Record the actual output and capture interface used during the lab run.

8. Comparison with Other Versions

Feature	T24 — Scapy	T25 — Raw socket
Packet decoding	Decoded layers are provided by Scapy.	Ethernet, IPv4, and TCP fields are parsed manually.
Portability	Works where Scapy and its capture backend are supported.	This code uses Linux AF_PACKET and is Linux-specific.
Maintenance	Less manual parsing code.	Must account for IP header length and packet structure.
Best learning point	High-level packet inspection.	How packet headers and protocol fields are represented in bytes.

9. Viva Questions and Answers

1. What is a raw socket?

A raw socket provides lower-level access to network packets than an ordinary application socket.

2. What is AF_PACKET?

On Linux, AF_PACKET allows a program to receive or send packets at the device/Ethernet layer.

3. What does EtherType 0x0800 mean?

It indicates that the Ethernet payload contains an IPv4 packet.

4. What is the IPv4 protocol value for TCP?

The IPv4 protocol field value 6 identifies TCP.

5. Why must the IP header length be checked?

IPv4 headers can contain optional fields, so the TCP header does not always begin at a fixed byte offset.

10. Common Errors: Message, Cause, and Fix

Message / symptom	Possible cause	Fix
Operation not permitted	The process lacks packet-capture privileges.	Run with sudo on an authorised Linux lab machine.
Address family not supported	The operating system does not support AF_PACKET.	Use Linux for this exact program; use a separately written platform-specific version on other systems.
No suspicious packets appear	Capture interface does not expose the tested traffic, or no packet uses a non-allowed port.	Verify the interface and test only on an authorised lab setup; localhost may not be visible to this program.
Index/struct parsing error	A truncated or malformed frame was received.	Keep length checks before reading each header field.